

Experiment No: 08

Experiment Name: Traversing, Searching, Insertion and Deletion in a Doubly Linked List and Circular Linked List

Objectives:

- To implement Doubly Linked List and Circular Linked List and perform operations such as traversal, searching, insertion and deletion
- To understand dynamic memory management, pointer manipulation and optimized data structures for efficient data handling

Theory:

A Doubly Linked List is a linear dynamic data structure where each node contains data, a pointer to the next node and a pointer to the previous node. Traversal can be performed in both forward and backward directions

A Circular Linked List is a variation where the last node points back to the head, allowing circular traversal. These structures allow dynamic insertion and deletion at any position efficiently compared to static arrays

Algorithms:

1.1 | Traversal (Doubly Linked List):

- Start from head
- Move forward using next pointer and display each node
- Move backward from last node using prev pointer

1.2 | Traversal (Circular Linked List):

- Start from head
- Move forward using next until reaching head again

2 | Searching:

- Start from head
- Compare each node's data with the target
- If found, display the position; otherwise, continue

3 | Insertion:

- Beginning: Update head and node pointers
- End: Traverse to last node, update pointers
- Specific Position: Traverse and adjust pointers to insert

4 | Deletion:

- Beginning: Update head, adjust pointers, free node
- End: Traverse to last node, update previous node's pointer, free node
- Specific Position: Traverse, update adjacent nodes, free node

Code:

Doubly Linked List:

```
@+ DLLcpp > _
1  #include <iostream>
2  using namespace std;
3
4  // Node structure for Doubly Linked List
5  struct Node
6  {
7      int data;
8      Node *next;
9      Node *prev;
10 };
11
12 Node *head = nullptr;
13
14 // Traverse in Doubly Linked List
15 void traverse()
16 {
17     Node *ptr = head;
18     if (ptr == nullptr)
19     {
20         cout << "List is empty\n";
21         return;
22     }
23     cout << "Forward: ";
24     while (ptr != nullptr)
25     {
26         cout << ptr->data << " ";
27         if (ptr->next == nullptr)
28             break;
29         ptr = ptr->next;
30     }
31     cout << "\nBackward: ";
32     while (ptr != nullptr)
33     {
34         cout << ptr->data << " ";
35         ptr = ptr->prev;
36     }
37     cout << "\n";
38 }
39
40 // Search in Doubly Linked List
41 void search(int key)
42 {
43     Node *ptr = head;
44     int pos = 0;
45     bool found = false;
46     while (ptr != nullptr)
47     {
48         if (ptr->data == key)
49         {
50             cout << "Element " << key << " found at position " << pos << "\n";
51             found = true;
52             break;
53         }
54         ptr = ptr->next;
55         pos++;
56     }
57     if (!found)
58         cout << "Element " << key << " not found\n";
59 }
60
61 // Insert at end of Doubly Linked List
62 void insertEnd(int val)
63 {
64     Node *newNode = new Node(val, nullptr, nullptr);
65     if (!head)
66     {
67         head = newNode;
68         return;
69     }
70     Node *ptr = head;
71     while (ptr->next != nullptr)
72         ptr = ptr->next;
73     ptr->next = newNode;
74     newNode->prev = ptr;
75 }
76
77 // Delete from end of Doubly Linked List
78 void deleteEnd()
79 {
80     if (!head)
81     {
82         cout << "List is empty\n";
83         return;
84     }
85     Node *ptr = head;
86     if (!ptr->next)
87     {
88         delete head;
89         head = nullptr;
90         return;
91     }
92     while (ptr->next != nullptr)
93         ptr = ptr->next;
94     ptr->prev->next = nullptr;
95     delete ptr;
96 }
97
98 int main()
99 {
100     int choice, val, key;
101     while (true)
102     {
103         cout << "\n~~~ Doubly Linked List Operations ~~~\n";
104         cout << "1. Traversal\n2. Search\n3. Insert at End\n4. Delete from End\n5. Exit\n";
105         cout << "Enter your choice: ";
106         cin >> choice;
107         switch (choice)
108         {
109             case 1:
110                 traverse();
111                 break;
112             case 2:
113                 cout << "Enter element to search: ";
114                 cin >> key;
115                 search(key);
116                 break;
117             case 3:
118                 cout << "Enter value to insert: ";
119                 cin >> val;
120                 insertEnd(val);
121                 break;
122             case 4:
123                 deleteEnd();
124                 break;
125             case 5:
126                 return 0;
127             default:
128                 cout << "Invalid choice! Try again\n";
129         }
130     }
131 }
```

Circular Linked List:

```
G++ CLC.cpp > _
1 #include <iostream>
2 using namespace std;
3
4 // Node structure for Circular Linked List
5 struct Node
6 {
7     int data;
8     Node *next;
9     Node *prev;
10 };
11
12 Node *head = nullptr;
13
14 // Traverse in Circular Linked List
15 void traverse()
16 {
17     if (!head)
18     {
19         cout << "List is empty\n";
20         return;
21     }
22     Node *ptr = head;
23     cout << "Forward: ";
24     do
25     {
26         cout << ptr->data << " ";
27         ptr = ptr->next;
28     } while (ptr != head);
29     cout << "\nBackward: ";
30     ptr = head->prev; // start from last node
31     Node *start = ptr;
32     do
33     {
34         cout << ptr->data << " ";
35         ptr = ptr->prev;
36     } while (ptr != start);
37     cout << "\n";
38 }
39
40 // Search in Circular Linked List
41 void search(int key)
42 {
43     if (!head)
44     {
45         cout << "List is empty\n";
46         return;
47     }
48     Node *ptr = head;
49     int pos = 0;
50     bool found = false;
51     do
52     {
53         if (ptr->data == key)
54         {
55             cout << "Element " << key << " found at position " << pos << "\n";
56             found = true;
57             break;
58         }
59         ptr = ptr->next;
60         pos++;
61     } while (ptr != head);
62     if (!found)
63         cout << "Element " << key << " not found\n";
64 }
65
66 // Insert at end of Circular Linked List
67 void insertEnd(int val)
68 {
69     Node *newNode = new Node(val, nullptr, nullptr);
70     if (!head)
71     {
72         head = newNode;
73         head->next = head;
74         head->prev = head;
75     }
76     else
77     {
78         Node *tail = head->prev; // last node
79         tail->next = newNode;
80         newNode->prev = tail;
81         newNode->next = head;
82         head->prev = newNode;
83     }
84 }
85
86 // Delete from end of Circular Linked List
87 void deleteEnd()
88 {
89     if (!head)
90     {
91         cout << "List is empty\n";
92         return;
93     }
94     if (head->next == head)
95     {
96         // only one node
97         delete head;
98         head = nullptr;
99     }
100     else
101     {
102         Node *tail = head->prev;
103         tail->prev->next = head;
104         head->prev = tail->prev;
105         delete tail;
106     }
107 }
108
109 int main()
110 {
111     int choice, val, key;
112     while (true)
113     {
114         cout << "\n~~~ Circular Linked List Operations ~~~\n";
115         cout << "1. Traversal\n2. Search\n3. Insert at End\n4. Delete from End\n5. Exit\n";
116         cout << "Enter your choice: ";
117         cin >> choice;
118         switch (choice)
119         {
120             case 1:
121                 traverse();
122                 break;
123             case 2:
124                 cout << "Enter element to search: ";
125                 cin >> key;
126                 search(key);
127                 break;
128             case 3:
129                 cout << "Enter value to insert: ";
130                 cin >> val;
131                 insertEnd(val);
132                 break;
133             case 4:
134                 deleteEnd();
135                 break;
136             case 5:
137                 return 0;
138             default:
139                 cout << "Invalid choice! Try again\n";
140         }
141     }
142 }
```

Procedure:

1. VS Code was opened and a new project was created
2. The Doubly Linked List code was written, compiled and syntax errors fixed
3. Menu driven operations such as Traversal, Search, Insert, Delete were performed and changes were verified after each action
4. Forward and backward traversal was observed to ensure correctness and memory was freed when nodes were deleted
5. The Doubly Linked List code was extended to the Circular Linked List by linking the last node to head and updating head->prev
6. The Circular Linked List operations were tested, ensuring traversal, insertion and deletion maintained proper circular links

Results:

Doubly Linked List:

```
~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 1

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 3

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2
Backward: 2 1

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2 3
Backward: 3 2 1

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 4
```

Circular Linked List:

```
~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 1

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 3

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2
Backward: 2 1

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2 3
Backward: 3 2 1

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 4
```

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Traversal (DLL/CLL)	$O(n)$	$O(1)$
Searching	$O(n)$	$O(1)$
Insertion at beginning	$O(1)$	$O(1)$ per node
Insertion at end	$O(n)$	$O(1)$ per node
Deletion at beginning	$O(1)$	$O(1)$
Deletion at end	$O(n)$	$O(1)$

Precautions:

- Always checked for NULL pointers before accessing nodes
- Validated positions for insertion and deletion to avoid segmentation faults
- Freed memory of deleted nodes to prevent memory leaks
- Avoided negative or invalid indices during operations

Limitations:

- Random access is inefficient compared to arrays
- Operations require pointer manipulation, which may lead to errors if not careful

Conclusion:

This experiment provided hands on experience in implementing and manipulating Doubly Linked Lists and understanding the logic for Circular Linked Lists. By completing this lab, we gain a solid understanding of advanced pointer based dynamic data structures, foundational for stacks, queues and complex structures like graphs

Experiment No: 09

Experiment Name: Traversing Binary Tree, Binary Search Tree, Insertion and Deletion on Binary Search Tree

Objectives:

- To implement Binary Tree traversal techniques (Inorder, Preorder and Postorder)
- To implement Binary Search Tree operations such as insertion and deletion

Theory:

A Binary Tree (BT) is a hierarchical data structure where each node has at most two children: left and right. It does not enforce any ordering on values

Tree traversal techniques are used to visit all nodes:

- Inorder (Left $\rightarrow$ Root $\rightarrow$ Right)
- Preorder (Root $\rightarrow$ Left $\rightarrow$ Right)
- Postorder (Left $\rightarrow$ Right $\rightarrow$ Root)

On the other hand, a Binary Search Tree (BST) is a special type of binary tree where:

- Left subtree contains smaller values than root
- Right subtree contains larger values than root

Algorithms:

1. Binary Tree Traversal:

1.1 | Inorder (Left $\rightarrow$ Parent $\rightarrow$ Right)

1.2 | Preorder (Parent $\rightarrow$ Left $\rightarrow$ Right)

1.3 | Postorder (Left $\rightarrow$ Right $\rightarrow$ Parent)

2. Binary Search Tree:

2.1 | Insertion:

- Start from root
- If tree is empty, create root node
- Compare value:
 - If smaller $\rightarrow$ go left
 - If larger $\rightarrow$ go right
- Insert at correct position

2.1 | Deletion:

- Search for node
 - Case 1: No child $\rightarrow$ delete directly
 - Case 2: One child $\rightarrow$ replace with child
 - Case 3: Two children $\rightarrow$ replace with inorder successor

Code:

1. Binary Tree Traversal

1.1 | Inorder

```
Lab 8 > G+ 1_inorder.cpp > inorder(Node *)
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45
46     return nodes[1];
47 }
48
49 void inorder(Node *root)
50 {
51     stack<Node *> st;
52     Node *curr = root;
53
54     while (curr != NULL || !st.empty())
55     {
56         while (curr != NULL)
57         {
58             st.push(curr);
59             curr = curr->left;
60         }
61         curr = st.top();
62         st.pop();
63         cout << curr->data << " ";
64         curr = curr->right;
65     }
66 }
67
68 int main()
69 {
70     Node *root = buildTreeFromFile("file.txt");
71     cout << "Inorder: ";
72     inorder(root);
73     cout << endl;
74     return 0;
75 }
```

1.2 | Preorder

```
Lab 8 > G+ 2_preorder.cpp > main()
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45
46     return nodes[1];
47 }
48
49 void preorder(Node *root)
50 {
51     if (!root)
52         return;
53     stack<Node *> st;
54     st.push(root);
55
56     while (!st.empty())
57     {
58         Node *curr = st.top();
59         st.pop();
60         cout << curr->data << " ";
61
62         if (curr->right)
63             st.push(curr->right);
64         if (curr->left)
65             st.push(curr->left);
66     }
67 }
68
69 int main()
70 {
71     Node *root = buildTreeFromFile("file.txt");
72     cout << "Preorder: ";
73     preorder(root);
74     cout << endl;
75     return 0;
76 }
```

1.3 | Postorder

Lab 8 >  3_postorder.cpp > ...

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45
46     return nodes[1];
47 }
48
49 void postorder(Node *root)
50 {
51     if (!root)
52         return;
53     stack<Node *> st1, st2;
54     st1.push(root);
55
56     while (!st1.empty())
57     {
58         Node *curr = st1.top();
59         st1.pop();
60         st2.push(curr);
61
62         if (curr->left)
63             st1.push(curr->left);
64         if (curr->right)
65             st1.push(curr->right);
66     }
67
68     while (!st2.empty())
69     {
70         cout << st2.top()->data << " ";
71         st2.pop();
72     }
73 }
74
75 int main()
76 {
77     Node *root = buildTreeFromFile("file.txt");
78     cout << "Postorder: ";
79     postorder(root);
80     cout << endl;
81     return 0;
82 }
```

Input File (file.txt):

Lab 8 >  file.txt

```
1  2 78 3
2  4 79 5
3  0 80 6
4  0 76 0
5  7 77 0
6  0 75 0
7  0 71 0
8  -1 -1 -1
```


2. Binary Search Tree

```
Lab 8 > G++ bst.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left, *right;
9
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *insertBST(Node *root, int val)
18 {
19     if (root == NULL)
20         return new Node(val);
21
22     if (val < root->data)
23         root->left = insertBST(root->left, val);
24     else if (val > root->data)
25         root->right = insertBST(root->right, val);
26
27     return root;
28 }
29
30 Node *minValueNode(Node *node)
31 {
32     while (node && node->left != NULL)
33         node = node->left;
34     return node;
35 }
36
37 Node *deleteBST(Node *root, int val)
38 {
39     if (root == NULL)
40         return root;
41
42     if (val < root->data)
43         root->left = deleteBST(root->left, val);
44     else if (val > root->data)
45         root->right = deleteBST(root->right, val);
46     else
47     {
48         if (root->left == NULL)
49         {
50             Node *temp = root->right;
51             delete root;
52
53             return temp;
54         }
55         else if (root->right == NULL)
56         {
57             Node *temp = root->left;
58             delete root;
59             return temp;
60         }
61
62         Node *temp = minValueNode(root->right);
63         root->data = temp->data;
64         root->right = deleteBST(root->right, temp->data);
65     }
66
67     return root;
68 }
69
70 void inorder(Node *root)
71 {
72     if (root != NULL)
73     {
74         inorder(root->left);
75         cout << root->data << " ";
76         inorder(root->right);
77     }
78 }
79
80 int main()
81 {
82     ifstream file("bst.txt");
83     int val;
84
85     Node *root = NULL;
86
87     while (file >> val)
88     {
89         if (val == -1)
90             break;
91         root = insertBST(root, val);
92     }
93
94     cout << "Inorder after insertion: ";
95     inorder(root);
96
97     root = deleteBST(root, 50);
98
99     cout << "\nInorder after deletion: ";
100    inorder(root);
101
102    return 0;
103 }
```

Input File (bst.txt):

```
Lab 8 > bst.txt
1  50 30 70 20 40 60 80 -1
```

Procedure:

1. IDE (VS Code) was opened and project created
2. Binary Tree traversal programs were executed using file input
3. Outputs of Inorder, Preorder, Postorder were verified
4. BST program was compiled and executed using separate file input
5. Insertion and deletion operations were tested
6. Outputs were verified using inorder traversal

Results:

Binary Tree:

Inorder: 76 79 71 77 78 80 75 Preorder: 78 79 76 77 71 80 75 Postorder: 76 71 77 79 75 80 78

Binary Search Tree:

Inorder after insertion: 20 30 40 50 60 70 80

Inorder after deletion: 20 30 40 60 70 80

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Tree Traversal	$O(n)$	$O(1)$
BST Insertion	$O(h)$	$O(1)$
BST Deletion	$O(h)$	$O(1)$

Here, h = height of tree

Precautions:

- Ensured correct file format before execution
- Checked NULL pointers before operations
- Avoided invalid node access during deletion
- Verified output after each operation

Limitations:

- BST may degrade to $O(n)$ in worst case
- File-based structure requires careful formatting
- No direct random access like arrays

Conclusion:

This experiment successfully demonstrated Binary Tree traversal and Binary Search Tree operations. Binary Tree traversal helped understand hierarchical structure navigation, while BST operations provided insight into efficient data organization using ordered insertion and deletion. These concepts are fundamental for advanced data structures and algorithms